



Chapter 4-System Modeling Requirements

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Why Goal-Oriented Requirements Engineering?**
 - **Goals provide the right abstractions for RE processes of**
 - **domain analysis**
 - **elicitation, elaboration**
 - **negotiation**
 - **specification**
 - **analysis**
 - **documentation**
 - **evolution**
 - **Good Model (GQ)** refers to quality attributes of the **best interface** between **multiple RE processes & actors**.
 - **GQ**= { Abstract, Understandable, Accurate, Inexpensive, Reusable }.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Why Goal-Oriented Requirements Engineering ...?**
 - A **goal model** is a **conceptual model** that documents **goals**, their **decomposition** into **sub-goals**, and existing **goal dependencies**.
 - A **conceptual model** is a representation of a system. It consists of concepts used to help people **know**, **understand**, or **simulate** a subject the **model** represents.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Why Goal-Oriented Requirements Engineering ...?**
 - A **goal** is a prescriptive statement of intent about some system (**existing** or **to be**) whose **satisfaction** in general requires the **cooperation** of **some of the agents** forming that system (=software and environment).
 - Goal-oriented requirements engineering (**GORE**) is concerned with the **use of goals** for eliciting, elaborating, structuring, specifying, analyzing, negotiating, documenting, and modifying requirements.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Why Goal-Oriented Requirements Engineering ...?**
 - **Domain properties** are descriptive statements about **an agent** in the environment. e.g., physical laws, organizational norms, etc.
 - **A requirement** is a **realizable goal** under **responsibility** of **an agent** in the **software-to-be** (expressed in terms of **monitored** and **controlled** objects).

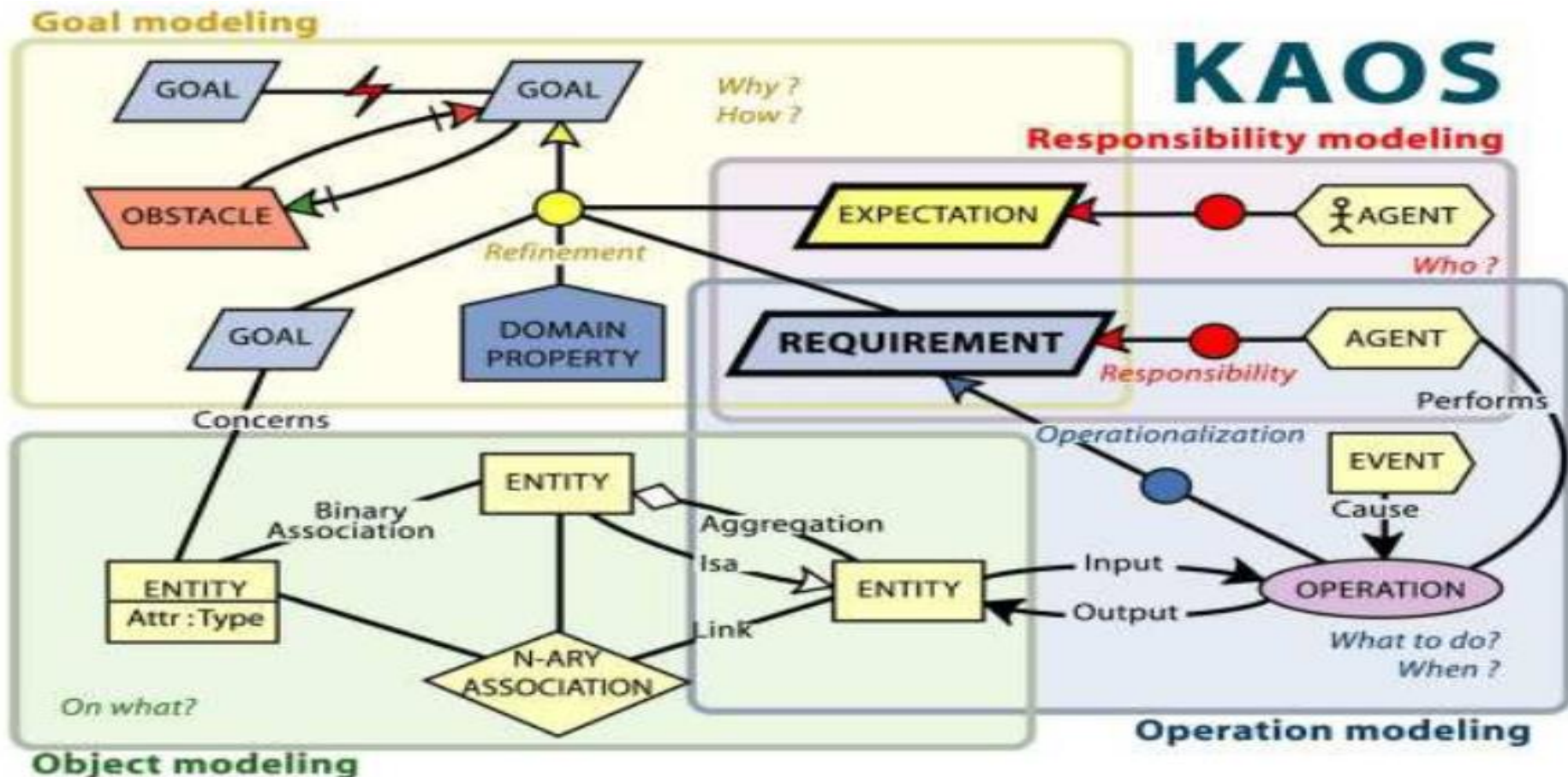
Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Why Goal-Oriented Requirements Engineering ...?**
 - An **expectation** is a **realizable goal** under **responsibility of an agent** in the environment (**unlike requirements, expectations cannot** be forced by **the system-to-be**).
 - An **operation** is an **input-output** relation over **objects**.
 - **The state of the system** (**software** or **environment**) is defined by **aggregation** of the **states of its objects**. An **object model** is represented by a **UML class diagram**.

Week 6, 7 and 8 Lessons: System Modeling Requirements

- **View of Goal Modeling Techniques (i*, KAOS):**
 - **GORE** combines Goal, Responsibility, Object and Operation Modeling.



Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **What are the rules to completed in the Goal-Oriented Requirements Engineering?**
 - When the five rules are satisfied, the requirements engineering, **in the strongest sense, is complete.**
 - **Rule-1: There is a set R of requirements. Each member of R has been validated (checked informally) as **acceptable** to the customer, and R as a whole has been **validated** as expressing all the customer's desires with respect to **the software development project.****

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **What are the rules to completed in the Goal-Oriented Requirements Engineering?** When the five rules ...
 - **Rule-2:** There is a set **K** of statements of domain knowledge. **Each member** of **K** has been validated (checked informally) as **true of the requirement**.
 - **Rule-3:** There is a set **S** of specifications. The members of **S** do not contain the environment they are not stated in terms of any **unshared actions** or **some components**, and they **do not refer to the future**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **What are the rules to completed in the Goal-Oriented Requirements Engineering?** When the five rules ...
 - **Rule-4: A proof shows that**
 - **S, K |- R**
 - This proof ensures that **an implementation** of **S and K** will **satisfy the requirements, R.**
 - **Rule-5: There is a proof that S and K are consistent.**
This ensures that the **specification** is **internally consistent** and **consistent** with the **environment.**

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **What should be satisfied in the Goal-Oriented Requirements Engineering?** When the five rules ...
 - **Note** that the **two proofs** together **imply** that **S**, **K**, and **R** are **consistent with each other**.
 - **Example: $P, C \mid - S$**
 - **If the machine is as described in C, the execution of the program P will ensure the behavior S at the machine's interface with the world.**

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Revision on **Modeling**: **a model** is an abstraction describing a system or a subset of a system.
 - Helps deal with **complexity**.
 - A **view** depicts the **selected aspects** of a model.
 - Each **view** focuses on a **subset of the model**.
 - **Example**: All the **blueprints** for a **house constitute** a model, but one **view** might focus just on the **plumbing schematics** (<https://www.geminivalve.com/plumbing-piping-diagram/>).
 - A **notation** is a set of **graphical** or **textual** rules for representing **views of models**.
 - **UML** (Unified Modeling Language) contains **notations** for **various types of models** in **object-oriented development**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Revision on **Application vs Solution Domain**
 - **Application Domain** - all aspects of the user's problem.
 - The environment that the system is to operate in.
 - Can change over time (includes **physical environment, people, work processes**, etc.).
 - Modeling and analysis need to understand things from the **user's point of view**.
 - Modeling it involves problem **specification** (or requirements elicitation) and **analysis**.
 - **Solution Domain** - modeling space of **all possible solutions** (systems)
 - Usually modeled in **much more detail** than **application domain**.
 - Need to be able to **translate solution model** into **code**.
 - Involves overall **system design**, as well as **object/module design**.
 - Developer's **point of view**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- Revision on **Primary Parts of a System's Model**
 - **Functional model**
 - Describes the *functionality* of a system from the user's point of view.
 - **Object Model**
 - Describes the structure of a **system** (and of **subsystems**) in terms of **objects**, **attributes**, **operations**, and **associations**.
 - **Dynamic Model**
 - describes the **internal behavior** of the system.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Revision on **UML - Unified Modeling Language**
 - Intended to provide **standard notations** for use **with object-oriented modeling**.
 - Attempts to integrate the best elements of **previous leading notations**.
 - Supported now by a number of **CASE** (Computer-Aided Software Engineering) tools.
 - Example: **StarUML, Visual Paradigm, UMLet, plantUML**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Revision on **UML - Unified Modeling Language ...**
 - Intended to provide **standard notations** for use **with object-oriented modeling**.
 - Attempts to integrate the best elements of **previous leading notations**.
 - Supported now by a number of **CASE** (Computer-Aided Software Engineering) tools.
 - Example: **StarUML, Visual Paradigm, UMLet, plantUML**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Revision on **UML Diagram Types**
 - UML 2.0 contains 13 different kinds of diagrams. The typical examples are as follows:
 - **Use Case Diagram**
 - Describes the functionality from the user's perspective. Used for the **functional model** of a system.
 - **Class/Object Diagram**
 - Describes the structure of a system or subsystem. Used in the **object model** of a system.
 - **Sequence Diagram**
 - A form of *interaction diagram*. Describes behavior as a sequence of messages exchanged among a set of objects. Used in the **dynamic model** of a system.
 - **State Machine (or Statechart) Diagram**
 - Describes behavior in terms of the possible states of a single object. Used in the **dynamic model** of a system.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- Revision on **Other UML Diagram Types**
 - **Activity Diagram** - help to represent **control flow and data flow** through a system.
 - **Package Diagram** - uses for grouping related **classes** and **objects** together.
 - **Deployment Diagram** - represents relationship between **software run-time** components and the **hardware nodes** of a system.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- Revision on **Modeling Notations types:**
 - Natural language
 - + No special training required
 - Ambiguous, verbose, vague, obscure .
 - No automation
 - Ad hoc notation (bubbles, squares and arrows)
 - + No special training required
 - No syntax formally defined meaning not clear, ambiguous
 - No automation- No automation

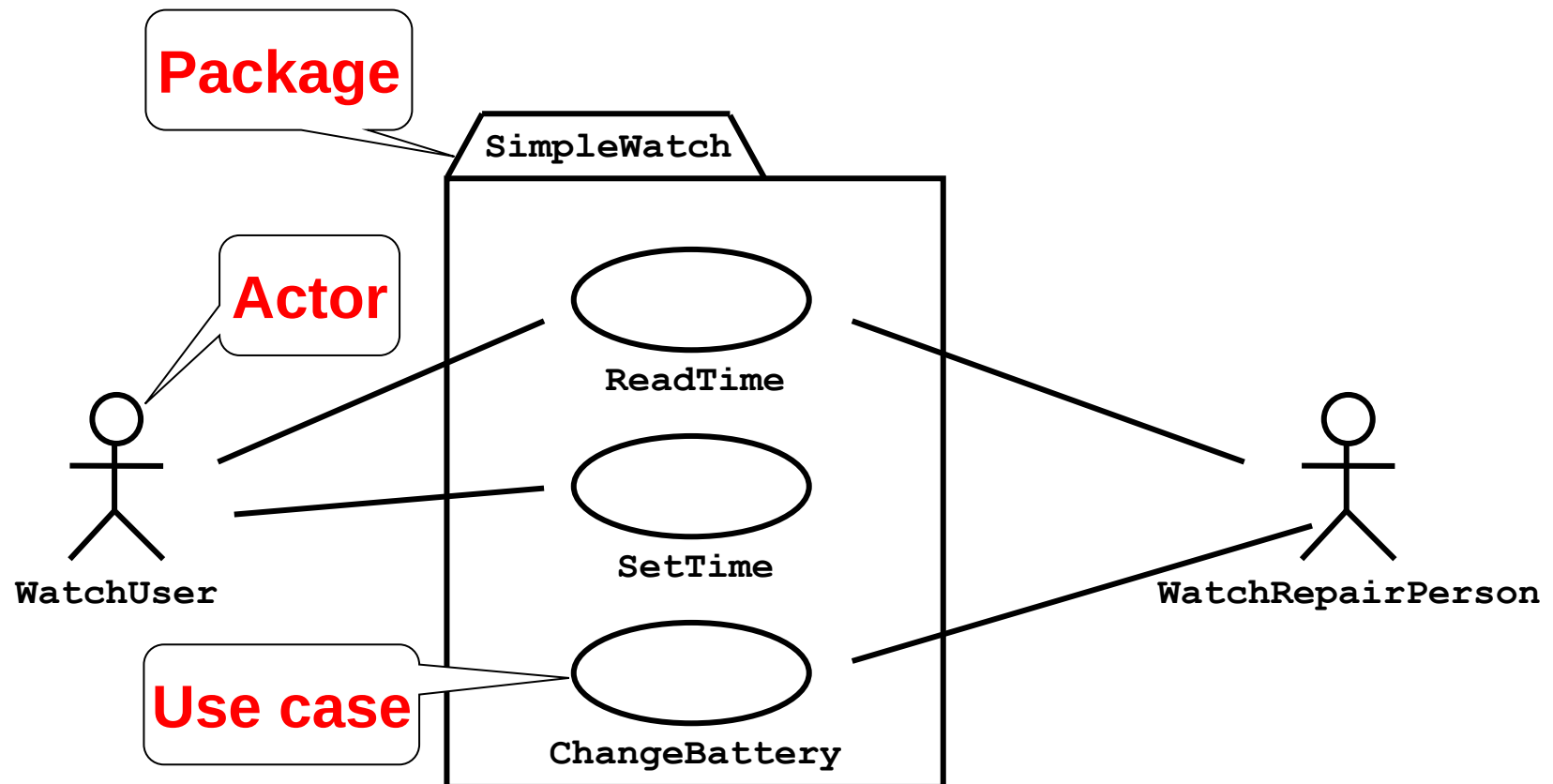
Week 6, 7 and 8 Lessons: System Modeling Requirements



- Revision on **Modeling Notations ...**
 - **Semi-formal** notation (URN, UML, SysML.)
 - + Syntax (graphics) well defined.
 - + Partial common understanding, reasonably easy to learn.
 - + Partial automation.
 - Meaning only defined informally.
 - Still a risk of ambiguities.
 - **Formal** notation (Logic, SDL, Petri nets, FSM ...)
 - + Syntax & semantics defined.
 - + Great automation (analysis and transformations).
 - More difficult to learn & understand.

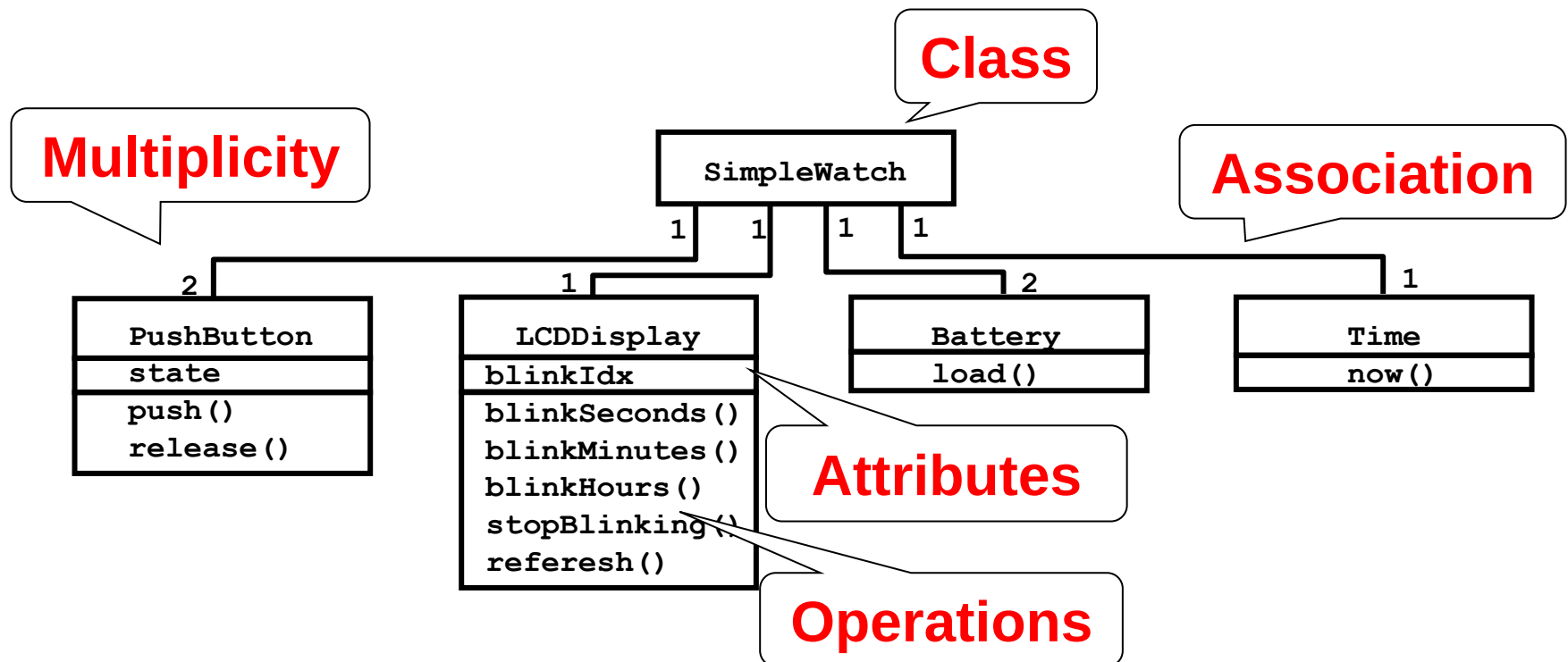
Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Example: Use Case Diagrams** represent the functionality of the system from **user's point of view**.



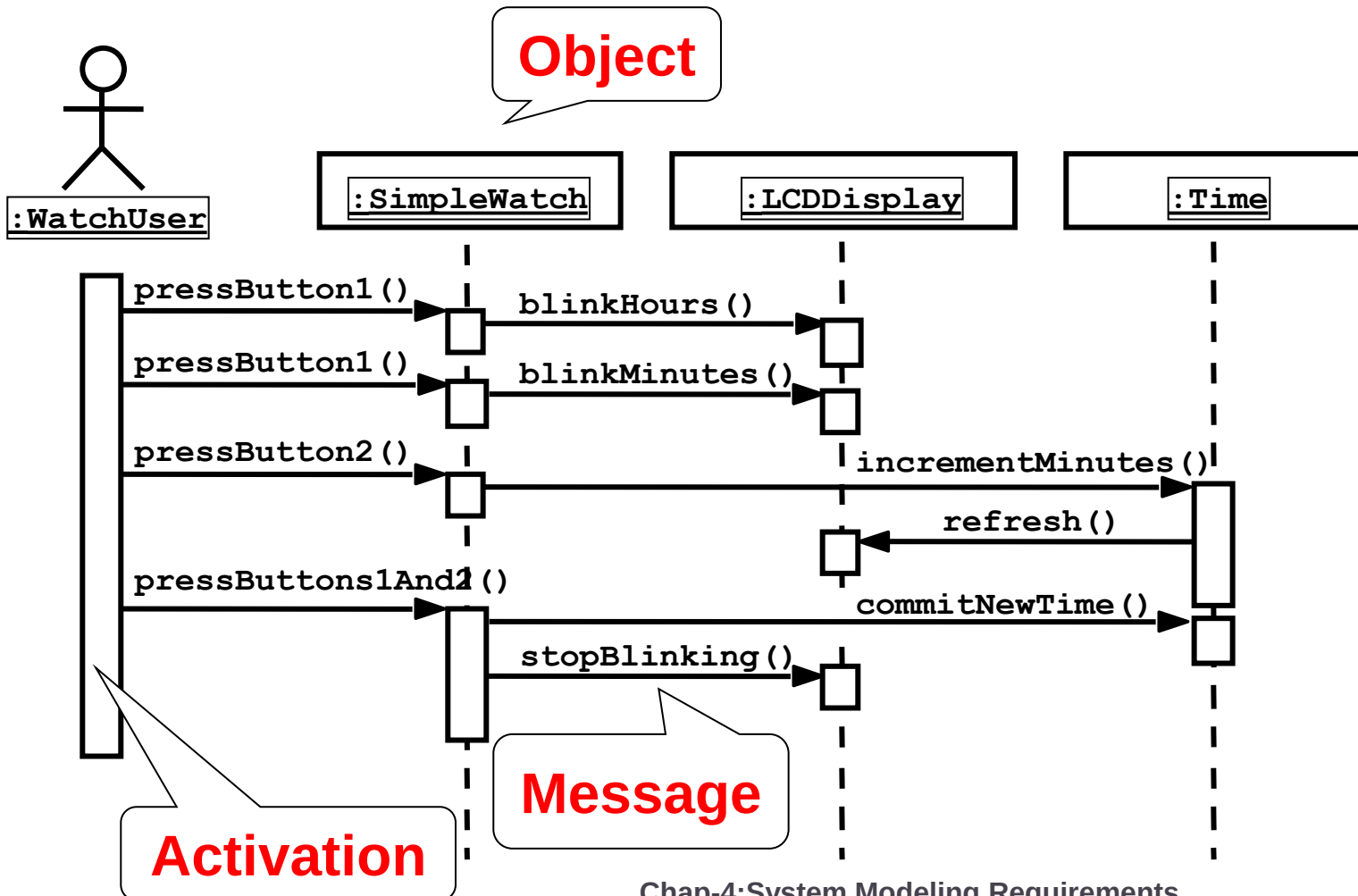
Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Example:** Class Diagrams represent the structure of the system.



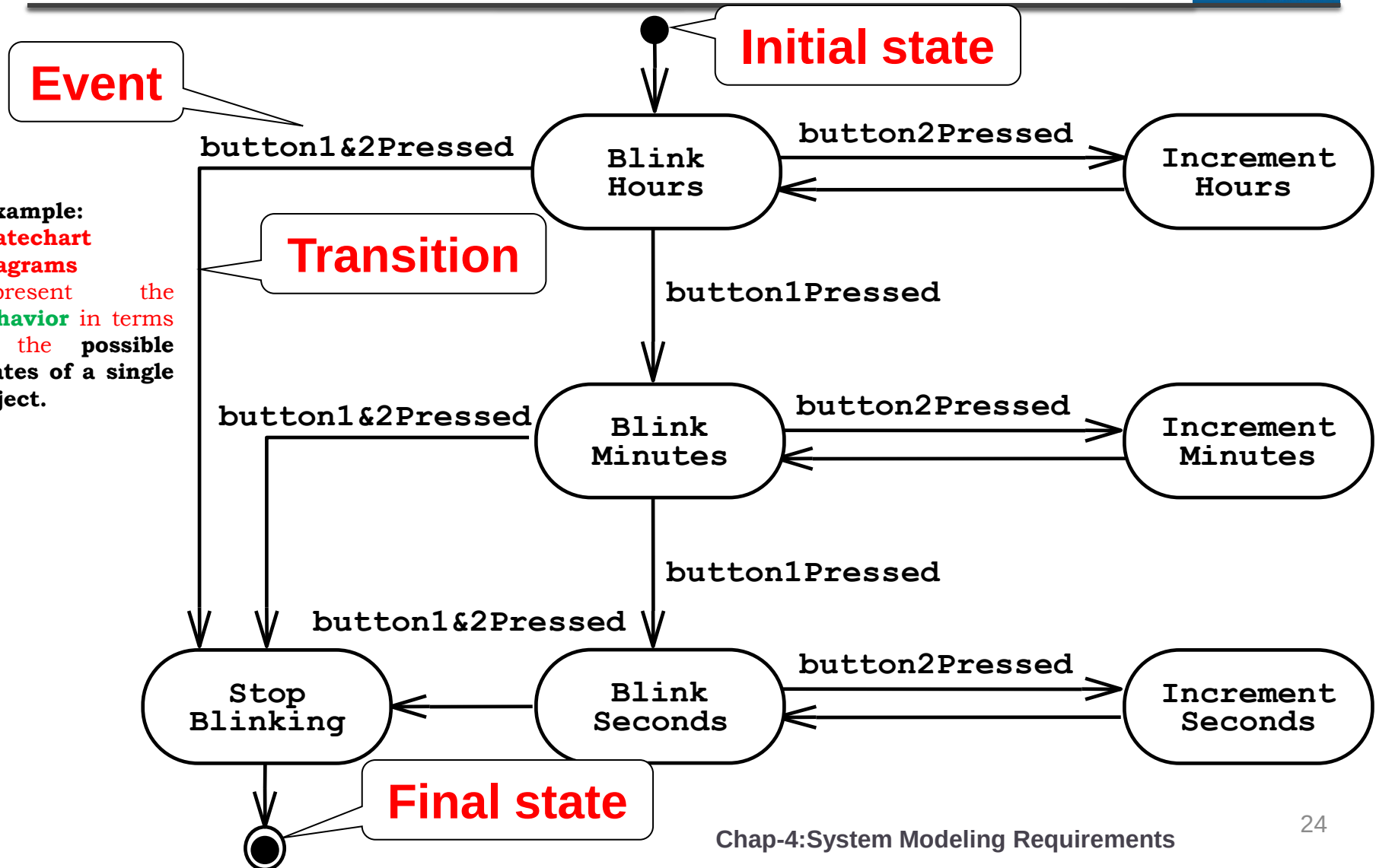
Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Example:** Sequence Diagram represent the behavior as interactions.





Week 6, 7 and 8 Lessons: System Modeling Requirements



Week 6, 7 and 8 Lessons:

System Modeling Requirements

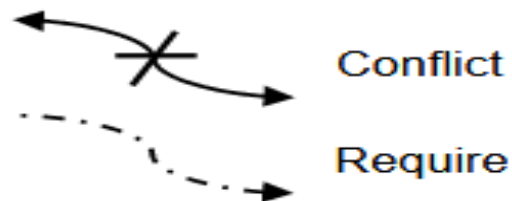
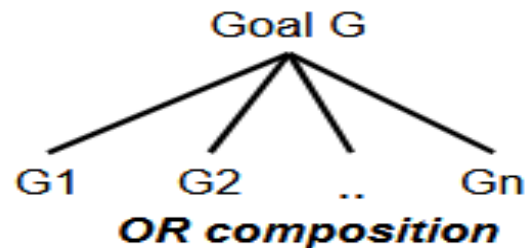
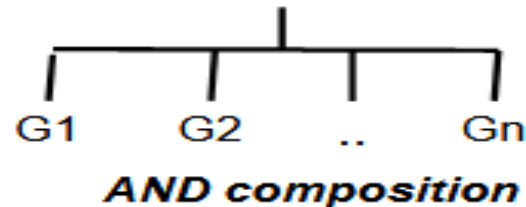


- **How to assess modeling? Using goal modeling techniques.**
 - **Common goal modeling language include different dialects:**
 - The Goal-oriented Requirements Language (GRL) [GRL 2009]
 - **KAOS** [Van Lamsweerde 2009] & **i*** [Yu 1993]
 - **Use Case Map via AND/OR trees decomposition.**
 - **Goal Modeling - Goal Oriented Requirements Engineering**
 - **A goal** is the intentional description of a desired characteristic of the **system to be**.
 - A goal is the **most abstract description of requirements**.
 - **All requirements** shall be **traced back** to a **goal**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **How to assess modeling? Using goal modeling techniques.**
 - **Goal Modeling Representation using AND /OR Trees**



Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Agent-Oriented Software Engineering using i* Framework**
 - **What ??**
 - *supports to develop a methodology for building agent oriented software model which supports **requirements analysis**, as well as **design**.*

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**

- **What is an Agent?**

- *A person, an organization, certain kinds of software.*
 - *An **agent** has **beliefs**, goals (**desires**), **intentions**.*
 - *Agents are situated, autonomous, flexible, and social.*
 - ***But note:** human/organizational agents can't be **prescribed**, they can only be **partially described**.*
 - ***Software agents**, on the other hand, have to be completely specified during **implementation**.*
 - ***Beliefs** correspond to (object) state, intentions constitute a run-time concept.*

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal Modeling Techniques (i*, KAOS)**
 - **Why Worry About Human/Organizational Agents?**
 - Because **their goals** lead to **software requirements**, and these **influence** the **design of a software system**.
 - Note **the role of human/organizational** agents in OOA **map** with **use cases**.
 - *In **KAOS**, **requirements analysis** begins with **a set of goals**; these are **analyzed/decomposed** to **simpler goals** which eventually either lead to **software requirements**, or are **delegated to external agents**.*

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **How to assess modeling? Using goal modeling techniques.**
 - **GRL :**
 - is a language for supporting **goal** and **agent oriented** modeling and **reasoning of requirements**, especially for dealing with **Non-Functional Requirements (NFRs)**.
 - provides constructs for expressing **various types of concepts** that appear during the **requirement** and **high-level architectural design** process.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - With GRL, there are **three** main categories of **concepts**: **intentional elements, links, and actors.**
 - The **intentional elements** in **GRL** are **goal, task, softgoal** and **resource.**
 - **GRL** contains the **intentional** concepts because they are used for **models** that allow **answering questions** such as
 - **why particular** and/or (**what alternatives**) behaviors, **informational** and **structural aspects** to be realized in the **system requirement.**

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - A **GRL model** can either **be composed of a global goal model**, or a **series of goal models distributed in several actors.**
 - If **a goal model** includes **more than one actor**, then the **intentional dependency relationships** between the **actors** can be **well represented** and **reasoned out.**
 - This implies, a **goal** is a **condition or state of affairs** in the world that the **stakeholders** intend to **achieve.**

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - A goal can be either a **business goal** or a **system goal**.
 - A **business goal** express **goals** regarding the **business or state of the business affairs the individual or organization** wishes to achieve.
 - **System goal** expresses **goals** the **target system** that describes the **functional requirements** of the target information system.
 - In GRL graphical representation, **goals** are represented as a **rounded rectangle** with **goal name inside**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - A **task** specifies a **particular way of doing something**.
 - **Task** refers to the **solutions** in the target system, which satisfies the **softgoals** (called **operationalization** in NFR) or achieve goals.
 - The **desired solutions** provide **operations, processes, data representations, structuring, constraints** and **agents** in the **target system** to meet the needs stated in the **goals** and **softgoals**.
 - In **GRL graphical** representation, **tasks** are represented as a **hexagon** with **task name inside**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - **A softgoal** is a **condition or state of affairs** in the world that the **actor** intends to achieve.
 - **Softgoal** is used to represent **NFRs** in the **future system**.
 - Non-functional requirements, such as **performance, security, accuracy, reusability, interoperability, time-to market** and **cost** are often **crucial for the success of a software systems**.
 - **NFRs** should be addressed **as early as possible** in a **software lifecycle**, and be properly reflected **in software architecture** before the **specific implementation is aware**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - **In GRL graphical representation, a softgoal** is represented an **irregular curvilinear shape** with **softgoal name inside**.
 - A **resource** is an (physical or informational) entity, with which the main concern is whether it is **available**.
 - **Resources** are shown as **rectangles** in GRL graphical representation.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - **The i* framework has two ways of representation:** the **graphical** and the **formal representations**.
 - As for the **graphical representation**, it has **two** main modeling components;
 - the **Strategic Dependency (SD) model** and
 - the **Strategic Rationale (SR) model**

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - **Strategic dependency (SD)** model-models relationships between the **actors**.
 - goal/softgoal **dependency**-an actor depends on another actor to attain a goal.
 - resource **dependency**-an actor needs a resource from another actor.
 - task **dependency**-an actor needs another actor to carry out a task.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - **Two parts to the model ...:**
 - **Strategic rationale (SR)** model-models concerns and interests of the actors.
 - Shows **task decompositions**
 - Shows **means-ends links** between **tasks** and **goals**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - **Links:** which connect the **i* model elements** together through **means-end**, **task-decomposition** or **contribution levels** (positive (Make/Help links), negative (Hurt/ Break links) and **the unknown**.
 - **The satisfaction levels.**
 - **Reasoning elements:** they are represented through **routines**, **rules** and **beliefs**.

Week 6, 7 and 8 Lessons:

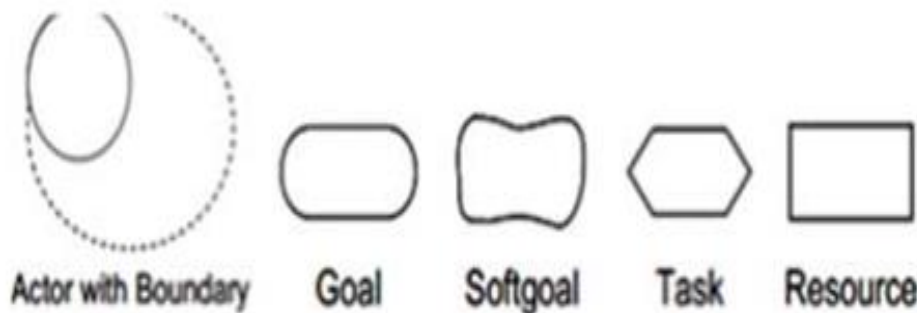
System Modeling Requirements



- **Goal-oriented Requirement Language (GRL) ...:**
 - The **i* basic elements** in the **GRL** are:
 - **Means-ends** is used to describe **how goals** are achieved.
 - Each **task** connected to a **goal** by **means-ends link** is an alternative means for achieving the goal.
 - **Decomposition** defines what other sub-elements needs to be achieved or available in order for a **task** to be **performed**.
 - **Contribution** describes how **softgoals, tasks, links** contribute to **others**.
 - A **contribution** is an effect that is a primary desire during modeling.
 - **Contributions** can be either **negative**, or **positive**, can be either **sufficient or partial**.

Week 6, 7 and 8 Lessons: System Modeling Requirements

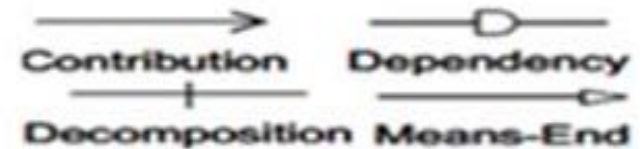
- **Goal-oriented Requirement Language (GRL) ...:**
 - The **i* basic elements symbols or notations** in the **GRL** are:



a. i* elements



c. i* contribution types



b. i* link types





Week 6, 7 and 8 Lessons: System Modeling Requirements

○ Goal-oriented Requirement Language (GRL) ...:

○ Concepts

○ Softgoal Interdependency Graph

- ☁ NFR Softgoal
- ⦿ Operationalizing Softgoal
- ☁ Claim Softgoal

○ Problem Interdependency Graph

- ⊖ Undesirable Situation
- ⊖ Undesirable Operationalization
- ⊖ Causal Attribution

Relationships

Decomposition Relationships

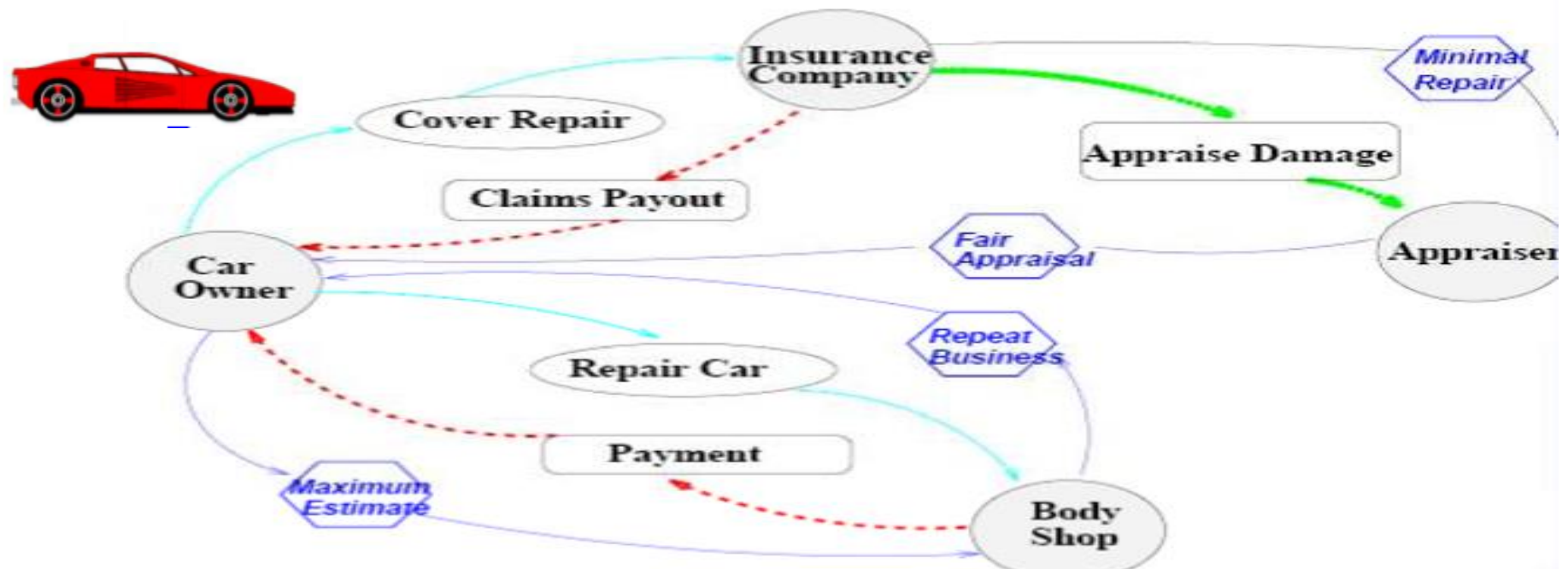
- ⊢ And Decomposition
- ⊢ Or Decomposition
- ⊢ Equal Decomposition

Contribution

- ⊢ Some +
- ⊢ Make
- ⊢ Help
- ⊢ Some -
- ⊢ Break
- ⊢ Hurt
- ⊢ Topic Reference

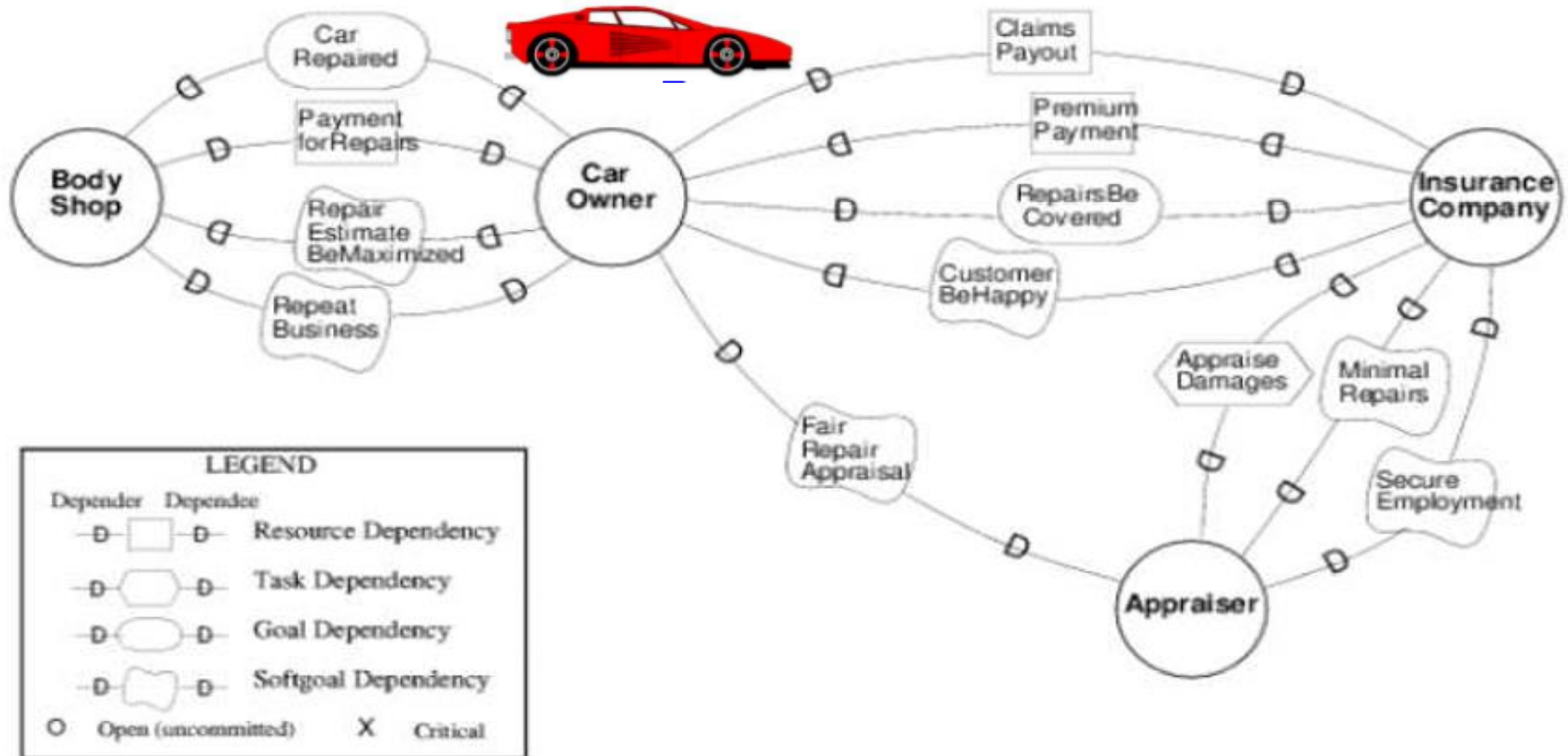
Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Goal-oriented Requirement Language (GRL) ...:**
 - **Example: Explicit representation of agents, roles, goals, tasks and resources.**
 - Why does it take so long to have a claim settled after an automobile accident?
 - Why does the company hire appraisers to assess damages?
 - What should a software agent do and why?



Week 6, 7 and 8 Lessons: System Modeling Requirements

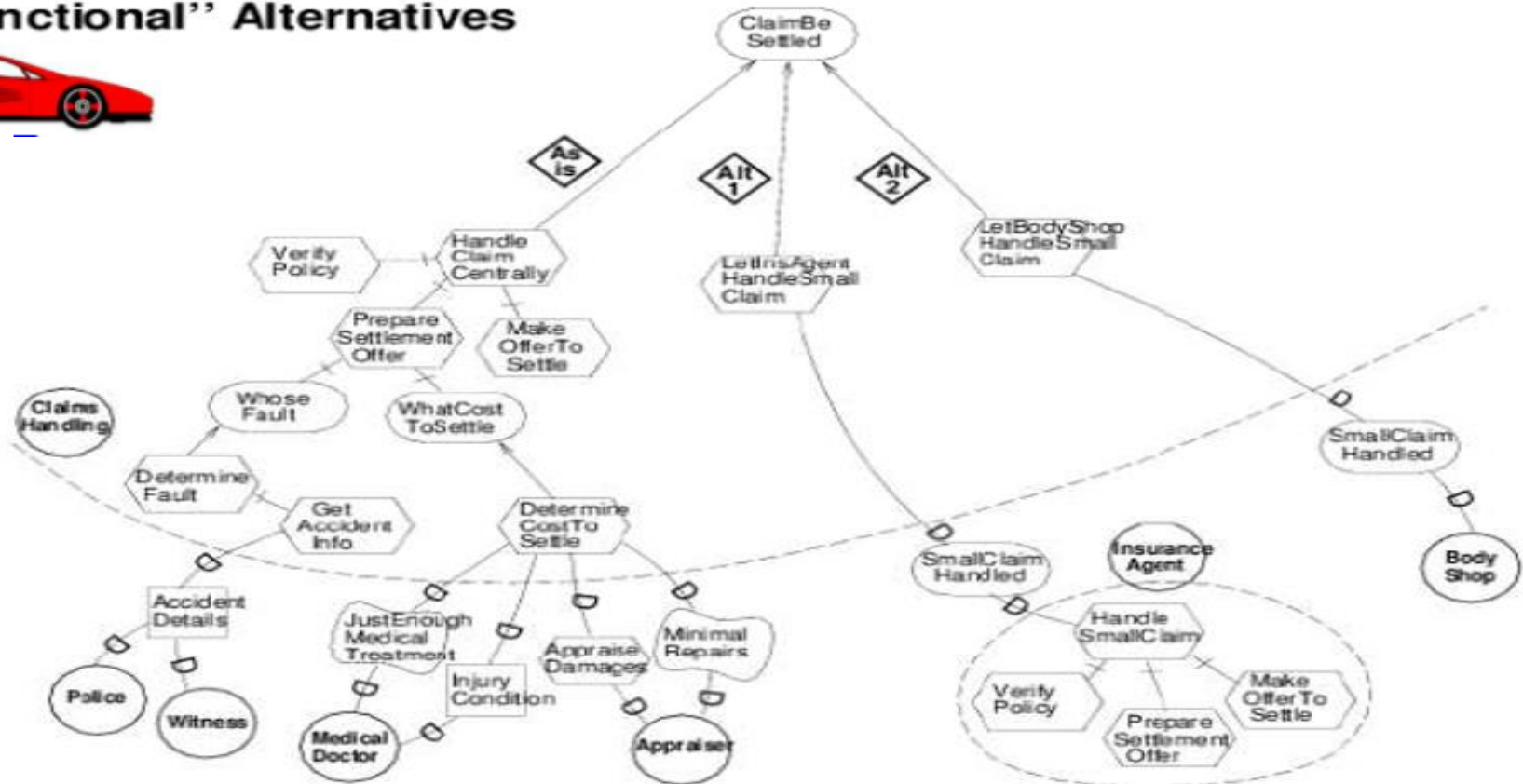
- **Goal-oriented Requirement Language (GRL) ...:**
 - **Strategic Dependency Model**



Week 6, 7 and 8 Lessons: System Modeling Requirements

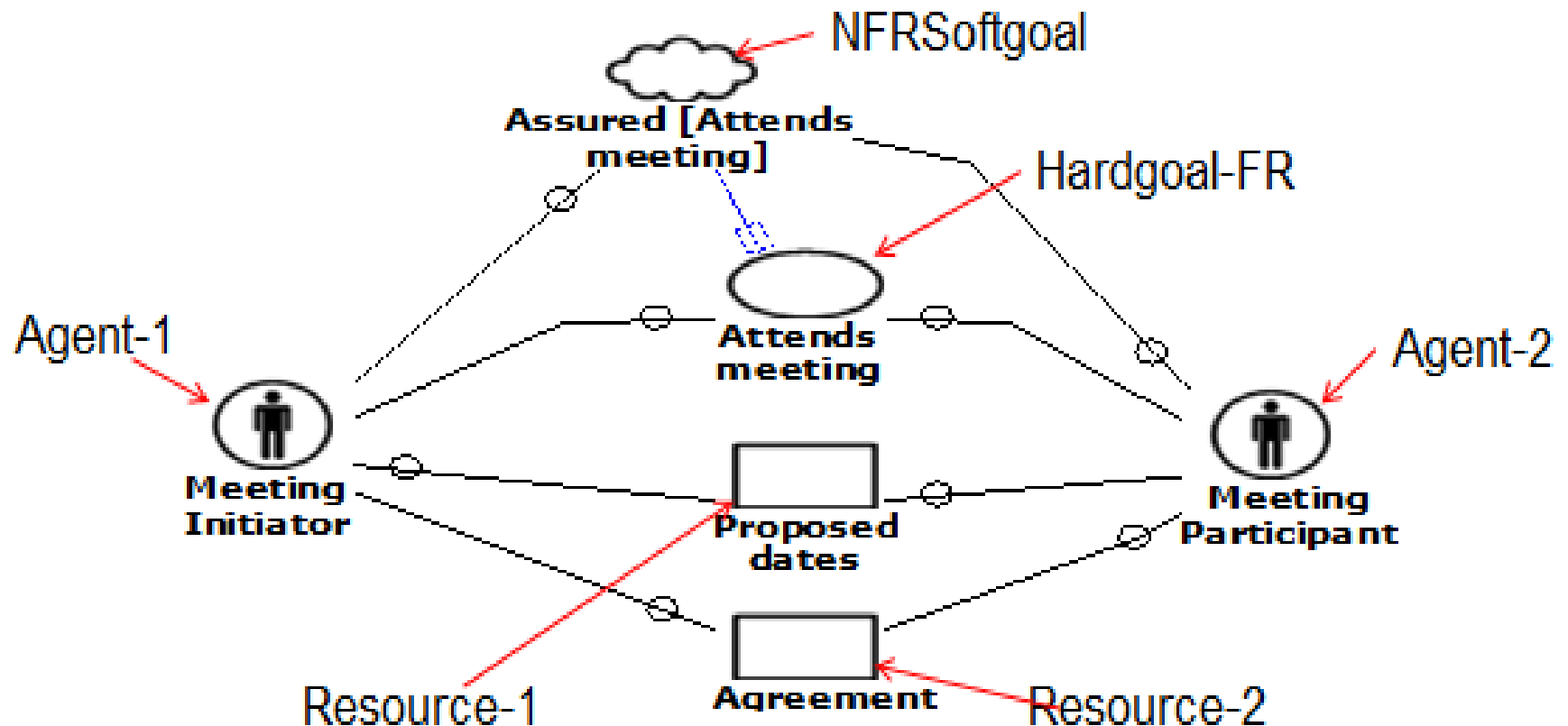
- **Goal-oriented Requirement Language (GRL) ...:**
 - **Strategic Rationale Model**

“Functional” Alternatives



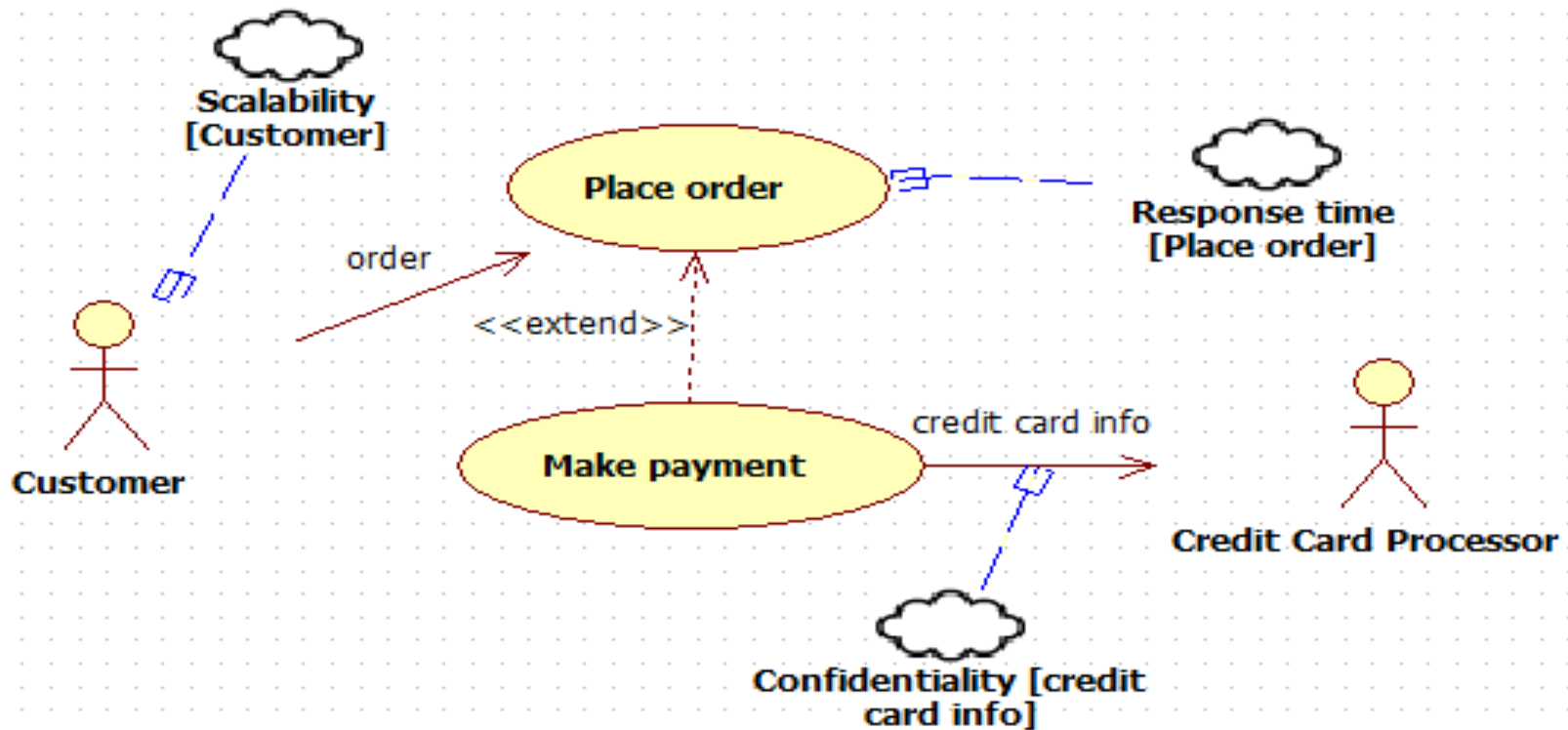
Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Goal Modeling Techniques (i*, KAOS)**
 - **Example: i* Framework for Meeting RE-Tools (with StarUML).**



Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Goal Modeling Techniques (i*, KAOS)**
 - **RE Use Case Model with StarUML RE Tools .**



Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Use Case Maps (UCM)**
 - provides a visual notation for scenarios, which is proposed for describing and reasoning about **large-grained behavior patterns in systems**, as well as the coupling of these patterns.
 - employs scenario paths to illustrate causal relationships among responsibilities.
 - also provides an integrated view of behavior and structure by allowing the superimposition of scenario paths on a structure of abstract components.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Use Case Maps (UCM)**
 - The combination of **behavior** and **structure** in UCMs enable **architectural reasoning**.
 - Scenarios in UCM can be **structured** and **integrated incrementally**.
 - This enables reasoning about and detection of potential **undesirable interactions** of **scenarios** and **components**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Use Case Maps (UCM)**
 - The **UCM notation** is mainly composed of **path elements**, and also **of components**.
 - The **basic path notation** address **simple operators** for **causally linking responsibilities** in **sequences**, as alternatives, and in **parallel**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Use Case Maps (UCM)**
 - Basic elements of **UCMs** are: **start points**, **responsibilities**, **end points** and **components**.
 - **Starting points** are **filled circles** representing **pre-conditions** or **triggering causes**.
 - **End points** are **bars** representing **post-conditions** or **resulting effects**.
 - **Responsibilities** are **crosses** representing **actions**, **tasks** or **functions** to be performed.
 - **Components** are **boxes** representing **entities** or **objects composing** the system.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Use Case Maps (UCM)**
 - Basic elements of UCMs are ...:
 - **Paths** are the **wiggle lines** that connect **start points**, **responsibilities** and **end points**.
 - A **responsibility** is said to be bound to a **component** when the **cross** is **inside the component**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Use Case Maps (UCM)**
 - **Alternatives** and **shared segments** of **routes** are represented as **overlapping paths**.
 - An **OR-join merges** two (or more) **overlapping paths** while an **OR-fork splits** a path into two (or more) **alternatives**.
 - **Alternatives** may be **guarded** by **conditions represented** as **labels** between **square brackets**.

Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Use Case Maps (UCM)**
 - Alternatives and shared segments of routes are represented as **overlapping paths**
 - **Concurrent** and **synchronized segments** of routes are represented through the **use of a vertical bar**.
 - An **AND-join synchronizes** two **paths together** while an **AND-fork splits** a path **into two** (or more) **concurrent segments**.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- **Use Case Maps (UCM)**
 - When **maps become too complex** to be represented as one **single UCM**, a mechanism for **defining** and **structuring sub-maps** become necessary.
 - A **top level UCM**, referred to as a **root map**, can include containers (called **stubs**) for **sub-maps** (called **plug-ins**).
 - **Stubs** are represented as **diamonds**.
 - **Stubs** and **plug-ins** are used to solve the problems of **layering** and **scaling** or the **dynamic selection** and **switching of implementation details**.

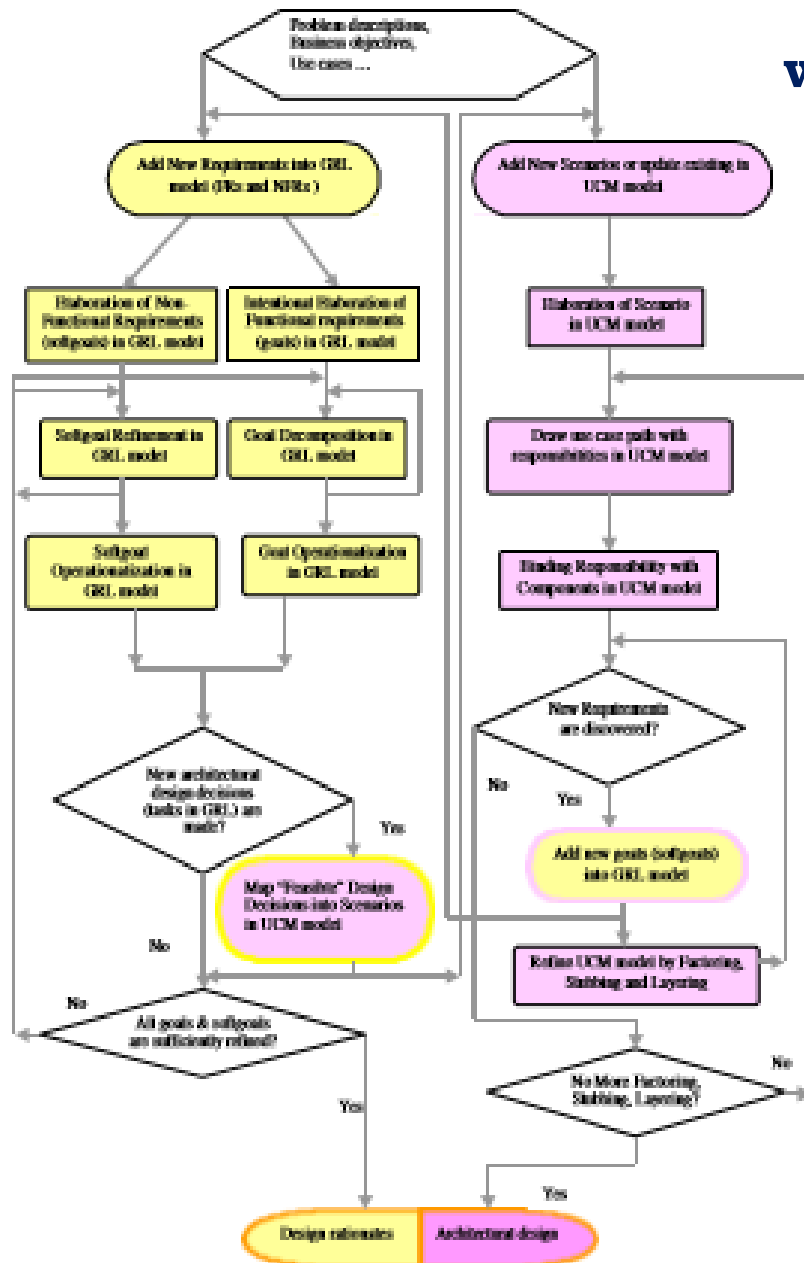
Week 6, 7 and 8 Lessons: System Modeling Requirements



- **Modeling Methodology with GRL+UCM**
 - **A complete requirement specification** should clarify the objectives **of a system to be** achieved, the **concrete behaviors** and **constraints** of the **system-to-be**, and the **entities** being **responsible** for certain **functions** in that system.

Modeling Methodology with GRL+UCM

- Integration of Goal-Oriented and Scenario-based Modeling.
- The **GRL-UCM** combination aims to elicit, refine and operationalize requirements incrementally until a satisfying architectural design is launched.



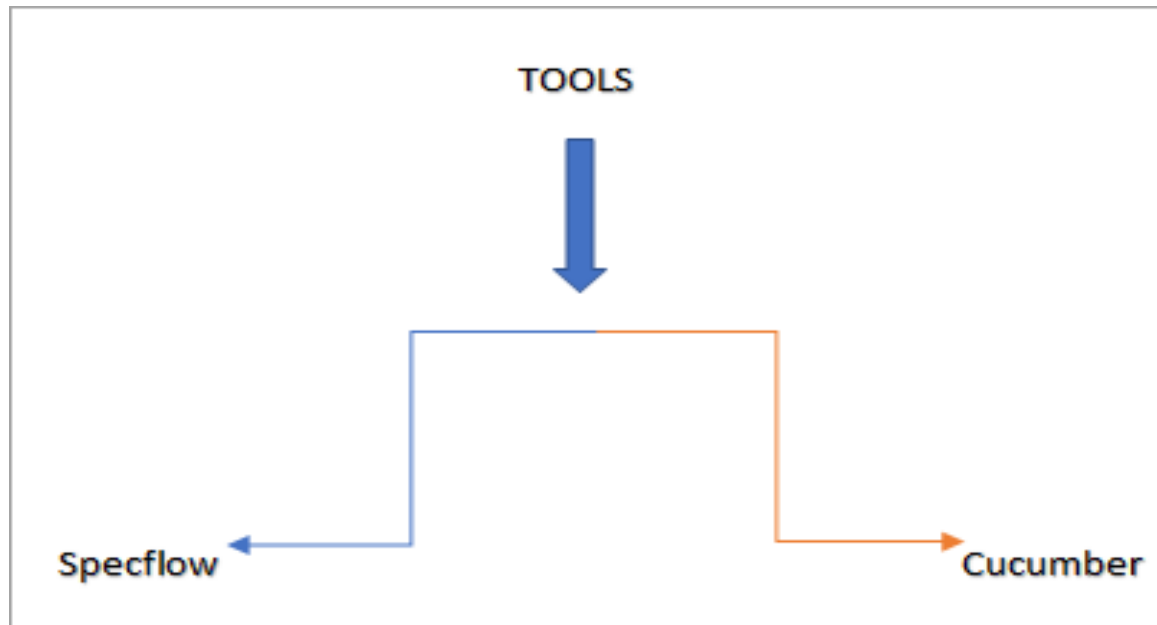


Week 6, 7 and 8 Lessons: System Modeling Requirements

- **Agile (without modeling) Approach:** Behavior-Driven Development
 - Behavior-Driven Development (**BDD**) is a software development approach that allows the tester/business analyst to create test cases in simple text language.
 - **Why Use BDD Framework?**
 - BDD provides a path that acts as a bridge to overcome the gap between the technical and the non-technical teams because the test cases are commonly written in simple text.
 - **How to Implement the BDD Approach?**
 - Test scenarios should be written in plain language with a detailed description of the test which supports the application and the application behavior to be understandable.

Week 6, 7 and 8 Lessons: System Modeling Requirements

- Behavior-Driven Development (BDD):
 - Typical BDD frameworks: Spec Flow and Cucumber.



- **Cucumber – A BDD Framework Tool**
 - Cucumber is a Behavior Driven Development (BDD) framework tool to write test cases.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Behavior-Driven Development (BDD):
 - **Terminology concepts**
 - **Cucumber** is one of the best tools which helps implement the Behavior Driven Development approach in the software project. This allows us to work with many technologies **E.g.** Java, Python, Jython, etc.
 - **Cucumber** is being widely used by many organizations and freelancers, it also has many communities where the users can discuss their issues and can easily find solutions to their problems.
 - **Cucumber language** – Gherkin which uses simple plain English words- reduces the communication gap between technical teams and stakeholders and allows them to work together at the same level.

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Behavior-Driven Development (BDD):
 - **Given – When – Then (GWT) Approach**
 - it is a structured format for expressing scenarios with example data, including pre- and post-conditions.
 - **Given:** Some given context (Preconditions).
 - **When:** Some Action is performed (Actions).
 - **Then:** Particular outcome/consequence after the above step (Results).

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Behavior-Driven Development (BDD):
 - **Given – When – Then (GWT) Approach**
 - **Sample Feature File**
 - **Feature:** BDD implementation using Cucumber
 - **Scenario:** Login to G-mail using Cucumber plugin
 - **Given** User is navigating to G-mail Login Page
 - **When** User need to enter username as "Username" and password as "Password"
 - **Then** User is successfully navigated to the G-mail Mail Box

```
Feature: BDD implementation using Cucumber
```

```
Scenario: Login to G-mail using Cucumber plugin
```

```
Given User is navigating to G-mail Login Page
```

```
When User need to enter username as "Username" and password as "Password"
```

```
Then User is successfully navigated to the G-mail Mail Box
```



Week 6, 7 and 8 Lessons:

System Modeling Requirements

- Behavior-Driven Development (BDD):
 - **Given – When – Then (GWT) Approach ...**
 - **Sample Step Definition File**

```
import cucumber.api.java.en.Given;  
import cucumber.api.java.en.Then;  
import cucumber.api.java.en.When;
```

```
public class Sample {  
    @Given("^User is navigating to G-mail Login Page$")  
    public void user is navigating to G mail Login Page() throws Throwable {  
        // Write code here that turns the phrase above into concrete actions  
    }  
    @When("^User need to enter username as \"([^\"]*)\" and password as \"([^\"]*)\" \"$")  
    public void user need to enter username as and password as(String arg1, String  
arg2) throws Throwable {  
        // Write code here that turns the phrase above into concrete actions  
    }  
    @Then("^User is successfully navigated to the G-mail Mail Box$")  
    public void user is successfully navigated to the G mail Mail Box() throws  
Throwable {  
        // Write code here that turns the phrase above into concrete actions  
    }  
}
```

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Behavior-Driven Development (BDD):
 - How GWT works. For a specific story, scenario and business rule, you analyze three things, asking and answering these questions:
 - **Given:** What is the context of the system? What pre-condition must be true before the action is tested? What data is in the system?
 - **When:** What will be tested? What input data will be tested via the business rule when the action occurs?
 - **Then:** In response to the action, what is the observable outcome? What are the expected results? What is the post-condition (state) or output data observable by the user?



Week 6, 7 and 8 Lessons: System Modeling Requirements

- Behavior-Driven Development (BDD):
 - **Example**

Story	As a customer I need to cancel a window cleaning job
Scenario	Customer contacts Squeaky Kleen two business days before the job's scheduled date
Business rule	A job canceled less than one business day before the Job's scheduled date must be charged a cancellation fee.
Given	
precondition(s)	Job status: scheduled
fixed data	Job number: 1255 Job scheduled date: 24 October 2012
When	
Action	Request to cancel a job
Input data	Request date: 22 October 2012
Then	
output data	Your job has been canceled. Would you like to reschedule for a future date?
postcondition	Job status: canceled



Week 6, 7 and 8 Lessons:

System Modeling Requirements

- Behavior-Driven Development (BDD):
 - With BDD, the 7 Product Dimensions represent categories of the product.
 - The beauty of the 7 Product Dimensions guide you toward a comprehensive understanding of the product.
 - In other words, conversing holistically across the 7 Product Dimensions helps you to discover product.

						
User	Interface	Action	Data	Control	Environment	Quality Attribute
Users interact with the product	The product connects to users, systems, and devices	The product provides capabilities for users	The product includes a repository of data and useful information	The product enforces constraints	The product conforms to physical properties and technology platforms	The product has certain properties that qualify its operation and development

Week 6, 7 and 8 Lessons:

System Modeling Requirements



- Behavior-Driven Development (BDD):
 - The position of each dimension on the canvas reflects the dimensions' relationships.
 - Here are some examples:
 - A user interacts with the product through interfaces.
 - An action operates on data.
 - A control may constrain data.
 - An action must be available (quality attribute) during specific times.
 - An interface must operate in an environment.